Weak Password Storage — Unsalted MD5

1. Report Title: Passwords Stored as Unsalted MD5

2. Finding ID: ELMS-004

3. Vulnerability Name: Use of a cryptographically broken, unsalted hash for password storage

4. Executive Summary

Every password in ELMS — employees and admin — is stored as a plain `md5()` digest with no salt and no work factor. MD5 is fast and broken; unsalted digests fall instantly to rainbow tables, and identical passwords produce identical hashes (visible in the data: all three seeded employees share one hash). This turns any hash disclosure (e.g., ELMS-003) into immediate plaintext credentials.

5. Severity

Medium (amplifier: becomes the pivot that makes ELMS-003 a full compromise).

6. CVSS 4.0

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N` scored in the context of a hash disclosure (ELMS-003) ? High; standalone the weakness is rated **Medium** as it requires hash access to exploit.

7. OWASP Classification

A02:2021 – Cryptographic Failures.

8. CWE Classification

CWE-916 (Use of Password Hash With Insufficient Computational Effort); CWE-759 (Use of One-Way Hash Without a Salt); CWE-327 (Broken/Risky Crypto Algorithm).

9. Affected URL

Server-side logic in `index.php` , `admin/index.php` , `forgot-password.php` , `emp-changepassword.php` , `admin/changepassword.php` , `admin/addemployee.php` .

10. Affected Parameter

`password` / `newpassword` (hashed with `md5()` before storage/comparison).

11. Authentication Requirements

N/A (storage design). Exploitation requires access to stored hashes (see ELMS-003).

12. Affected User Roles

All (employees and admin).

13. Vulnerability Description

Authentication and password-change code hashes passwords with `md5($_POST['password'])` and stores/compares that digest. No per-user salt, no adaptive algorithm (bcrypt/argon2), no pepper. The seeded data demonstrates the weakness directly: three employees with the same password have an identical stored hash.

14. Technical Root Cause

Use of a fast, unsalted, cryptographically broken digest for password storage instead of a memory-hard adaptive KDF.

15. Preconditions

To recover plaintext, an attacker needs the hashes — obtainable unauthenticated via ELMS-003, or via any future DB exposure/SQLi in another build.

16. Proof of Concept

The disclosed admin hash `5c428d8875d2948607f3e3fe134d71b4` equals `md5("Test@12345")` , and the employee hash `f925916e2754e5e03f75dd58a5733251` equals `md5("Test@123")` — verified locally:

```
md5('Test@123')   = f925916e2754e5e03f75dd58a5733251   (matches dump)
md5('Test@12345') = 5c428d8875d2948607f3e3fe134d71b4   (matches dump)
```

17. Reproduction Steps

1. Obtain a hash (e.g., from `elmsdb.sql` , ELMS-003).
2. Compute `md5()` of candidate passwords or look the hash up in any public MD5 database / rainbow table.
3. Observe the plaintext is recovered instantly (both seeded hashes resolve to the known passwords above).

18. HTTP Request Evidence

Not an HTTP-triggered flaw; evidence is the source (`$password=md5($_POST['password']);`) and the hash-match proof above (evidence log event `validation-complete`).

19. HTTP Response Evidence

See ELMS-003 response for the disclosed hashes that this finding renders crackable.

20. Screenshots

Not applicable (algorithmic/storage finding); the hash-match computation is the evidence.

21. Security Impact

Any hash disclosure yields plaintext passwords for all accounts, including admin. Unsalted hashing also reveals which users share passwords and enables precomputed-table attacks at scale.

22. Exploitation Feasibility

Very high once hashes are known; MD5 of common passwords is effectively instant to reverse.

23. Existing CVE Verification

No product CVE — this is a design weakness, not a versioned vulnerability. Status: **No matching CVE identified / not applicable** .

24. References

- OWASP Password Storage Cheat Sheet; A02:2021; CWE-916/759/327; NIST SP 800-63B §5.1.1.2.

25. Remediation

Replace MD5 with `password_hash()` (bcrypt or Argon2id) and verify with `password_verify()` . Migrate transparently: on next successful login, re-hash the supplied password with the strong algorithm. Enforce a password policy and consider a server-side pepper.

26. Recommended Secure Code Example

```
// Storing / changing a password
$hash = password_hash($_POST['newpassword'], PASSWORD_ARGON2ID);
$dbh->prepare("UPDATE tblemployees SET Password=:p WHERE id=:id")
    ->execute([':p'=>$hash, ':id'=>$id]);

// Verifying at login (+ transparent upgrade from legacy md5)
$row = /* fetch by email */;
```

```
$ok = (strlen($row->Password) === 32 && ctype_xdigit($row->Password))
      ? hash_equals($row->Password, md5($_POST['password']))   // legacy row
      : password_verify($_POST['password'], $row->Password);
if ($ok && password_needs_rehash($row->Password, PASSWORD_ARGON2ID)) {
    $new = password_hash($_POST['password'], PASSWORD_ARGON2ID);
    /* UPDATE ... SET Password=$new */
}
```

27. Retest Procedure

Register/reset a password, read its stored value, and confirm it is an Argon2id/bcrypt string (`$argon2id$…` / `$2y$…`), not a 32-hex MD5, and that two users with the same password have different stored hashes.

28. Final Conclusion

Confirmed by source and by matching the disclosed hashes to known plaintexts. Medium standalone; the multiplier that upgrades ELMS-003 to full compromise. Move to an adaptive, salted KDF.